

配置实验项目调试环境

编译原理课程组
中国科学技术大学

- **GDB 简介及安装**
- GDB 命令行调试
- GDB 结合VS Code图形化调试

• GDB简介:

- GDB(GNU Debugger) 是GNU软件系统中的标准调试器, 目前GDB所能支持的调试语言有C, C++等。GDB具备各种调试功效, 能针对计算机程序的执行进行追踪与警告, 使用GDB的调试人员可以监督及修改程序的内部变量值

• GDB安装 **演示**

- 在虚拟机的 terminal 中输入
sudo apt install gdb
- 验证:

```
hj@hj:~$ gdb --version
GNU gdb (Ubuntu 12.1-0ubuntu1~22.04) 12.1
Copyright (C) 2022 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
```

- GDB 简介及安装
- **GDB 命令行调试**
- GDB 结合VS Code图形化调试

- **GDB命令行调试 在终端直接使用GDB调试代码。**

- **步骤一 生成可执行文件： 演示**

- 调试项目C文件：CMake记录了项目文件的依赖关系，需要在项目根目录的CMakeLists.txt设置Debug模式，修改完成后重新编译整个项目。

```
set(CMAKE_BUILD_TYPE "Debug")
```

- **步骤二： gdb调试可执行文件 演示**

- gdb 可执行文件
- set args 参数1 参数2

```
(gdb) set args -emit-llvm ../gcd.cminus  
(gdb) █
```

- GDB命令行调试
- 步骤三：list查看代码并打断点

```
(gdb) list
28     bool gvn = false;
29     bool dump_json = false;
30     bool emit = false;
31     bool assembly = false;
32
33     for (int i = 1; i < argc; ++i) {
34         if (argv[i] == "-h"s || argv[i] == "--help"s) {
35             print_help(argv[0]);
36             return 0;
37         } else if (argv[i] == "-o"s) {
(gdb) b 33
Note: breakpoint 2 also set at pc 0x55555555d9df.
Breakpoint 3 at 0x55555555d9df: file /home/hj/compiler_cmiinus_ta/src/cminusfc
```

- 步骤四：执行到断点

```
(gdb) r
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /home/hj/compiler_cmiinus_ta/build/cminusfc -llvm-emit ../gdb.cminus
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 2, main (argc=3, argv=0x7fffffffe128) at /home/hj/compiler_cmiinus_ta/src/c
33     for (int i = 1; i < argc; ++i) {
```

- 步骤五：查看变量并继续执行并结束

```
(gdb) p i
$4 = 0
(gdb) c
Continuing.
Usage: /home/hj/compiler_cmiinus_ta/build/cminusfc [ -h | --help ] [ -o <target-file> ]
[Inferior 1 (process 2306) exited normally]
```

GDB命令调试代码还有很多，就不一一列举，同学们按需学习GDB调试代码

- GDB 简介及安装
- GDB 命令行调试
- **GDB 结合VS Code图形化调试**

在VS Code中配置图形化调试界面



命令行调试程序相对繁琐，通过vscode中安装C++/C插件图形化界面能帮助同学们更好调试程序

• 步骤零 安装C/C++插件：演示

使用快捷键 Ctrl+Shift+X 呼出扩展面板

在搜索框中输入：C/C++

再安装由 Microsoft 提供的名为 C/C++ 插件。

1. 先点击扩展按钮，切换到扩展面板
2. 在搜索栏搜索 C/C++ 插件
3. 找到由 Microsoft 发布的 C/C++ 插件，点击安装即可

在VS Code中配置图形化调试界面

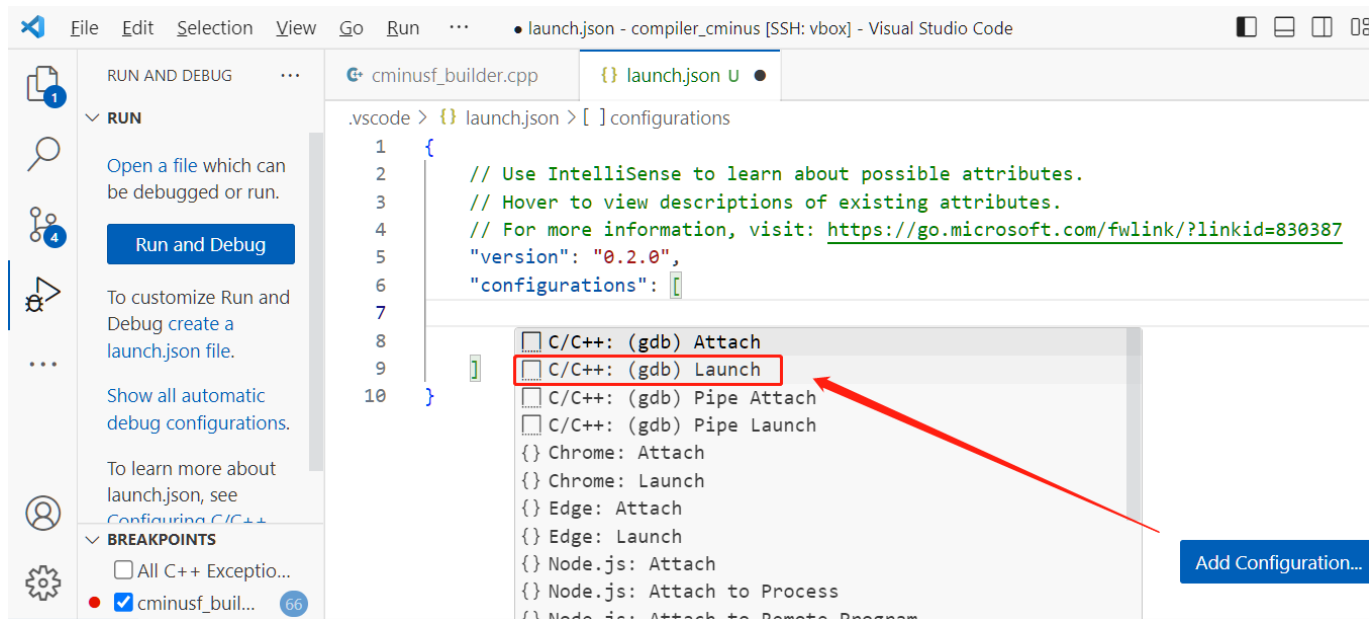


• 步骤一：生成可执行文件

调试项目C文件：CMake记录了项目文件的依赖关系，需要在项目根目录的CMakeLists.txt设置Debug模式，修改完成后重新编译整个项目。

• 步骤二：生成json文件

点击Run and Debug（快捷键：Ctrl + Shift + D），在.vscode/中新建launch.json。如图在launch.json文件中点击右下角的Add Configuration，选择gdb Launch。



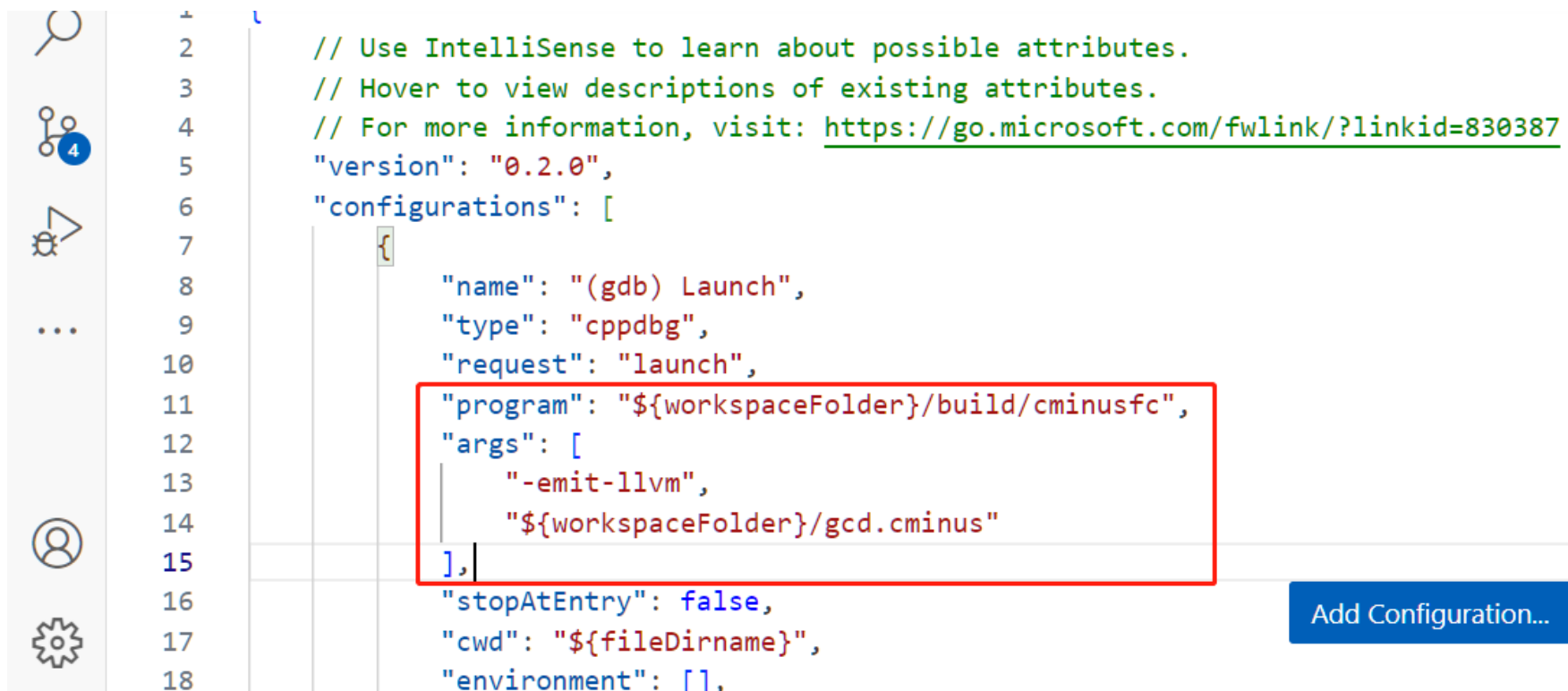
在VS Code中配置图形化调试界面



• 步骤三：配置gdb json文件

选择可执行文件的路径，以及启动参数。“program”为待调试程序，“args”为启动该程序的参数，在该图参数是“-emit-llvm”与“\${workspaceFolder}/gcd.cminus”

“\${workspaceFolder}”指的是VS Code中打开文件夹的路径)



```
1 // Use IntelliSense to learn about possible attributes.
2 // Hover to view descriptions of existing attributes.
3 // For more information, visit: https://go.microsoft.com/fwlink/?linkid=830387
4 "version": "0.2.0",
5 "configurations": [
6   {
7     "name": "(gdb) Launch",
8     "type": "cppdbg",
9     "request": "launch",
10    "program": "${workspaceFolder}/build/cminusfc",
11    "args": [
12      "-emit-llvm",
13      "${workspaceFolder}/gcd.cminus"
14    ],
15    "stopAtEntry": false,
16    "cwd": "${fileDirname}",
17    "environment": [],
```

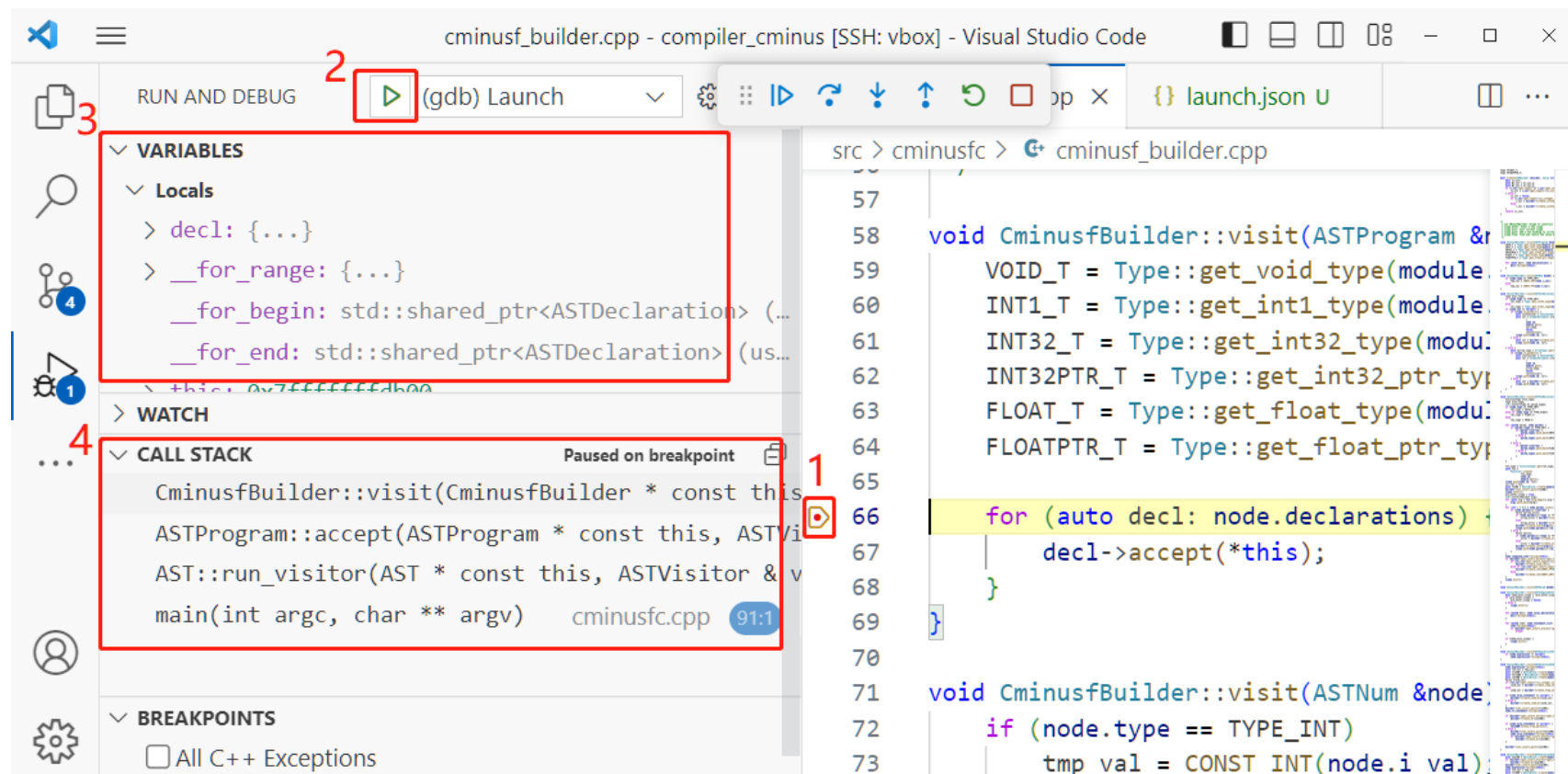
Add Configuration...

在VS Code中配置图形化调试界面



• 步骤四：图形化调试代码

在1处设置完断点，点击2处的三角运行符号，程序将运行至断点处停下，在3处显示变量的值，在4处显示的是目前函数的调用堆栈可以帮助理解函数的调用过程。





下期再见！

Thanks!